

OS Lab Cheatsheet

Threading · Semaphores · CPU Scheduling · Deadlock
CSE4502 Operating Systems Lab

Race Condition

Why it happens: `counter++` is 3 CPU steps: READ → ADD → WRITE. Two threads can interleave these steps and both read the same value, so one increment is lost. The final value is always less than 200000.

Listing 1: Race Condition — two threads increment a shared counter

```

1  #include <stdio.h>
2  #include <pthread.h>
3
4  int counter = 0;           // shared resource    no protection!
5
6  void* increment(void* arg) {
7      for (int i = 0; i < 100000; i++)
8          counter++;        // CRITICAL SECTION: READ-ADD-WRITE (3 steps, not atomic)
9      return NULL;
10 }
11
12 int main() {
13     pthread_t t1, t2;
14     pthread_create(&t1, NULL, increment, NULL); // launch thread 1
15     pthread_create(&t2, NULL, increment, NULL); // launch thread 2
16     pthread_join(t1, NULL); // main waits for t1 to finish
17     pthread_join(t2, NULL); // main waits for t2 to finish
18     printf("Final counter: %d\n", counter); // expected 200000, actual < 200000
19     return 0;
20 }
```

Mutex (Mutual Exclusion Lock)

Why mutex: A mutex is a lock. Only ONE thread holds it at a time. Others block (wait) at `pthread_mutex_lock()` until the holder calls `pthread_mutex_unlock()`. Wrap the critical section between lock and unlock to make it atomic.

Multi-Account Transfer (Lock Ordering to Prevent Deadlock)

Why per-account mutexes: One global mutex serializes everything. Per-account mutexes allow threads touching different accounts to run in parallel.

Why lock ordering: If T1 locks A then waits for B, and T2 locks B then waits for A ⇒ circular wait ⇒ deadlock. Fix: always lock smaller index first.

Listing 2: Mutex — Multi-Account Bank Transfer

```

1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define A 0
5  #define B 1
6  #define C 2
7  #define NUM_ACCOUNTS 3
8
9  int balance[NUM_ACCOUNTS] = {1000, 1000, 1000};
10 const char *name[] = {"A", "B", "C"};
11 pthread_mutex_t acc_mutex[NUM_ACCOUNTS]; // one lock per account
12 pthread_mutex_t log_mutex;              // one lock for printing
13
14 int transfer(int src, int dst, int amount, const char *tname) {
15     // DEADLOCK PREVENTION: always lock smaller index first
16     int first = (src < dst) ? src : dst;
17     int second = (src < dst) ? dst : src;
18
19     pthread_mutex_lock(&acc_mutex[first]); // lock both accounts
20     pthread_mutex_lock(&acc_mutex[second]); // before touching any balance
21
22     int ok = 0;
23     if (balance[src] >= amount) { // no negative balance allowed
24         balance[src] -= amount;
25         balance[dst] += amount;
26         ok = 1;
27     }
```

```

27     pthread_mutex_lock(&log_mutex);
28     printf("[%s] %d: %s->%s | A=%d B=%d C=%d\n",
29           tname, amount, name[src], name[dst],
30           balance[A], balance[B], balance[C]);
31     pthread_mutex_unlock(&log_mutex);
32 }
33 pthread_mutex_unlock(&acc_mutex[second]);
34 pthread_mutex_unlock(&acc_mutex[first]);
35 return ok;
36 }
37
38 void *thread_T1(void *arg) { for(int i=0;i<10;i++) transfer(A,B,100,"T1"); return NULL; }
39 void *thread_T2(void *arg) { for(int i=0;i<20;i++) transfer(B,C,50, "T2"); return NULL; }
40 void *thread_T3(void *arg) { for(int i=0;i<40;i++) transfer(C,A,25, "T3"); return NULL; }
41
42 int main() {
43     for(int i=0;i<NUM_ACCOUNTS;i++) pthread_mutex_init(&acc_mutex[i], NULL);
44     pthread_mutex_init(&log_mutex, NULL);
45     pthread_t t1,t2,t3;
46     pthread_create(&t1,NULL,thread_T1,NULL);
47     pthread_create(&t2,NULL,thread_T2,NULL);
48     pthread_create(&t3,NULL,thread_T3,NULL);
49     pthread_join(t1,NULL); pthread_join(t2,NULL); pthread_join(t3,NULL);
50     printf("Final: A=%d B=%d C=%d (sum=%d)\n",
51           balance[A],balance[B],balance[C],balance[A]+balance[B]+balance[C]);
52     for(int i=0;i<NUM_ACCOUNTS;i++) pthread_mutex_destroy(&acc_mutex[i]);
53     return 0;
54 }

```

Semaphore

Two uses of semaphores:

1. **Capacity limiter** (init > 0): `sem_wait` = take a slot, `sem_post` = release a slot.
2. **Signal/relay** (init = 0): `sem_wait` = freeze until signalled, `sem_post` = send the signal.

Limited-Capacity Server (Semaphore as Capacity Limiter)

Listing 3: Semaphore — 6 clients, server handles max 3 at once

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  #define NUM_CLIENTS 6
7  sem_t      server_sem;           // counting semaphore, init = 3
8  int        served[NUM_CLIENTS]; // status board
9  pthread_mutex_t status_mutex;    // protects the status board
10 int        all_done = 0;
11
12 void *monitor(void *arg) {        // prints status every 400ms
13     while (!all_done) {
14         pthread_mutex_lock(&status_mutex);
15         for (int i=0;i<NUM_CLIENTS;i++)
16             printf("C%d=%s%s", i+1, served[i]?"True ":"False",
17                   i<NUM_CLIENTS-1?" ", "\n");
18         pthread_mutex_unlock(&status_mutex);
19         usleep(400000);
20     }
21     return NULL;
22 }
23
24 void *client(void *arg) {
25     int id = *(int*)arg;
26     sem_wait(&server_sem);         // BLOCK if 3 already inside (counter=0)
27     pthread_mutex_lock(&status_mutex);
28     served[id] = 1;               // mark as being served
29     pthread_mutex_unlock(&status_mutex);
30     sleep(1);                     // simulate service
31     pthread_mutex_lock(&status_mutex);
32     served[id] = 0;
33     pthread_mutex_unlock(&status_mutex);
34     sem_post(&server_sem);        // release slot, wake a waiting client
35     return NULL;
36 }
37

```

```

38 int main() {
39     sem_init(&server_sem, 0, 3);          // 3 slots available
40     pthread_mutex_init(&status_mutex, NULL);
41     pthread_t monitor_tid, tids[NUM_CLIENTS];
42     int ids[NUM_CLIENTS];
43     pthread_create(&monitor_tid, NULL, monitor, NULL);
44     for(int i=0; i<NUM_CLIENTS; i++){ids[i]=i; pthread_create(&tids[i], NULL, client, &ids[i]);}
45     for(int i=0; i<NUM_CLIENTS; i++) pthread_join(tids[i], NULL);
46     all_done=1; pthread_join(monitor_tid, NULL);
47     sem_destroy(&server_sem); return 0;
48 }

```

Ordered Startup (Semaphore as Signal — init = 0)

Why init=0: A semaphore starting at 0 means “red light — wait here until someone posts.” This chains threads into a strict sequence regardless of which thread starts first.

Listing 4: Semaphore — Ordered startup: Power → Comm → Control

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  sem_t sem_power_done; // Power posts -> Comm waits
7  sem_t sem_comm_done;  // Comm posts -> Control waits
8
9  void *power_thread(void *arg) {
10     sleep(1);
11     printf("[Power] Initialized. Signal sent.\n");
12     sem_post(&sem_power_done); // green light for Comm
13     return NULL;
14 }
15 void *comm_thread(void *arg) {
16     printf("[Comm] Waiting...\n");
17     sem_wait(&sem_power_done); // FREEZE until Power signals
18     sleep(1);
19     printf("[Comm] Initialized. Signal sent.\n");
20     sem_post(&sem_comm_done); // green light for Control
21     return NULL;
22 }
23 void *control_thread(void *arg) {
24     printf("[Control] Waiting...\n");
25     sem_wait(&sem_comm_done); // FREEZE until Comm signals
26     sleep(1);
27     printf("[Control] Initialized.\n");
28     return NULL;
29 }
30 int main() {
31     sem_init(&sem_power_done, 0, 0); // both start as red lights
32     sem_init(&sem_comm_done, 0, 0);
33     pthread_t tc, tcomm, tp;
34     pthread_create(&tc, NULL, control_thread, NULL); // starts first but runs last
35     pthread_create(&tcomm, NULL, comm_thread, NULL);
36     pthread_create(&tp, NULL, power_thread, NULL); // starts last but runs first
37     pthread_join(tp, NULL); pthread_join(tcomm, NULL); pthread_join(tc, NULL);
38     sem_destroy(&sem_power_done); sem_destroy(&sem_comm_done);
39     return 0;
40 }

```

Readers-Writers Problem

Why read_count: Multiple readers can read simultaneously — only the FIRST reader needs to block writers (acquires write_block), and only the LAST reader releases writers. Readers in the middle don’t touch the writer lock at all. mutex protects read_count itself from concurrent modification.

Listing 5: Readers-Writers — multiple readers OR one writer at a time

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  int shared_data = 0;
7  int read_count = 0;
8  sem_t mutex; // protects read_count

```

```

9  sem_t write_block; // blocks writers when readers active (and vice versa)
10
11 void *reader(void *arg) {
12     int id = *(int*)arg;
13     printf("[Reader %d] Waiting...\n", id);
14     // Entry: first reader locks out writers
15     sem_wait(&mutex);
16     read_count++;
17     if (read_count == 1) sem_wait(&write_block); // first reader blocks writers
18     sem_post(&mutex);
19
20     printf("[Reader %d] Reading: %d\n", id, shared_data); // READ
21     sleep(1);
22
23     // Exit: last reader unlocks writers
24     sem_wait(&mutex);
25     read_count--;
26     if (read_count == 0) sem_post(&write_block); // last reader frees writers
27     sem_post(&mutex);
28     return NULL;
29 }
30
31 void *writer(void *arg) {
32     int id = *(int*)arg;
33     printf("[Writer %d] Waiting...\n", id);
34     sem_wait(&write_block); // need exclusive access blocks if any reader active
35     printf("[Writer %d] Writing...\n", id);
36     sleep(1);
37     shared_data++;
38     printf("[Writer %d] Updated to %d\n", id, shared_data);
39     sem_post(&write_block); // release exclusive access
40     return NULL;
41 }
42
43 int main() {
44     sem_init(&mutex, 0, 1);
45     sem_init(&write_block, 0, 1);
46     pthread_t r[3], w[2];
47     int rids[3]={1,2,3}, wids[2]={1,2};
48     for(int i=0;i<3;i++) pthread_create(&r[i],NULL,reader,&rids[i]);
49     pthread_create(&w[0],NULL,writer,&wids[0]);
50     usleep(100000);
51     pthread_create(&w[1],NULL,writer,&wids[1]);
52     for(int i=0;i<3;i++) pthread_join(r[i],NULL);
53     for(int i=0;i<2;i++) pthread_join(w[i],NULL);
54     return 0;
55 }

```

Dining Philosophers (Deadlock Prevention via Semaphore)

Why N-1 seats: With 5 philosophers and 5 forks, if all 5 sit and grab their left fork simultaneously $\Rightarrow$ circular wait $\Rightarrow$ deadlock. Allowing only 4 to sit guarantees by the pigeonhole principle that at least one philosopher has both forks available.

Listing 6: Dining Philosophers — semaphore limits seating to N-1

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  #define N 5
7  sem_t table_sem; // allows max N-1 = 4 philosophers seated
8  pthread_mutex_t fork_mutex[N]; // one mutex per fork
9
10 void *philosopher(void *arg) {
11     int id = *(int*)arg;
12     int left = id, right = (id+1)%N;
13     for (int meal=0; meal<3; meal++) {
14         printf("[Phil %d] Thinking\n", id);
15         sleep(1);
16         sem_wait(&table_sem); // sit: block if 4 already seated
17         pthread_mutex_lock(&fork_mutex[left]); // pick up left fork
18         pthread_mutex_lock(&fork_mutex[right]); // pick up right fork
19         printf("[Phil %d] EATING (meal %d)\n", id, meal+1);
20         sleep(1);
21         pthread_mutex_unlock(&fork_mutex[right]);

```

```

22     pthread_mutex_unlock(&fork_mutex[left]);
23     sem_post(&table_sem);           // stand up: free seat
24 }
25 return NULL;
26 }
27 int main() {
28     sem_init(&table_sem, 0, N-1); // KEY: N-1 seats, not N
29     for(int i=0;i<N;i++) pthread_mutex_init(&fork_mutex[i],NULL);
30     pthread_t threads[N]; int ids[N];
31     for(int i=0;i<N;i++){ids[i]=i; pthread_create(&threads[i],NULL,philosopher,&ids[i]);}
32     for(int i=0;i<N;i++) pthread_join(threads[i],NULL);
33     sem_destroy(&table_sem);
34     return 0;
35 }

```

H2O Bond Formation (Barrier using Semaphores)

Why 3 semaphores: `sem_h` counts arrived H atoms (O waits on it twice). `sem_o` signals H atoms that O is ready (each H waits on it once). `sem_bond` synchronises all 3 so they “bond” and exit simultaneously. Nobody leaves until the complete group (2H + 1O) is assembled.

Listing 7: H2O Barrier — atoms wait until complete H-H-O group forms

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  sem_t sem_h;    // counts H atoms that arrived
7  sem_t sem_o;    // O uses this to release H atoms
8  sem_t sem_bond; // synchronises final simultaneous exit
9
10 void *hydrogen(void *arg) {
11     int id = *(int*)arg;
12     printf("H%d Ready\n", id);
13     sem_post(&sem_h);    // announce: I (H) have arrived
14     sem_wait(&sem_o);    // FREEZE: wait for O to give green light
15     printf("H%d Bonding\n", id);
16     sem_post(&sem_bond); // signal: I am ready to bond
17     sem_wait(&sem_bond); // wait for all 3 to be ready
18     printf("H%d Bonded!\n", id);
19     return NULL;
20 }
21
22 void *oxygen(void *arg) {
23     printf("O Ready\n");
24     sem_wait(&sem_h);    // wait for first H
25     sem_wait(&sem_h);    // wait for second H
26     printf("O Bonding\n");
27     sem_post(&sem_o);    // release H1
28     sem_post(&sem_o);    // release H2
29     sem_wait(&sem_bond); // wait for H1 ready
30     sem_wait(&sem_bond); // wait for H2 ready
31     printf("\n>>> Bond Formed! (H-H-O) <<<\n\n");
32     return NULL;
33 }
34 int main() {
35     sem_init(&sem_h, 0, 0); sem_init(&sem_o, 0, 0); sem_init(&sem_bond, 0, 0);
36     pthread_t ht[4], ot[2]; int hids[4]={1,2,3,4};
37     pthread_create(&ht[0],NULL,hydrogen,&hids[0]);
38     pthread_create(&ot[0],NULL,oxygen, NULL);
39     pthread_create(&ht[1],NULL,hydrogen,&hids[1]);
40     usleep(100000);
41     pthread_create(&ht[2],NULL,hydrogen,&hids[2]);
42     pthread_create(&ot[1],NULL,oxygen, NULL);
43     pthread_create(&ht[3],NULL,hydrogen,&hids[3]);
44     for(int i=0;i<4;i++) pthread_join(ht[i],NULL);
45     for(int i=0;i<2;i++) pthread_join(ot[i],NULL);
46     return 0;
47 }

```

Histogram Sharding (Local then Merge)

Why local first: If every thread updates the global histogram directly, they'd fight over the lock constantly (high contention). Instead: each thread counts independently into its own private array (no lock needed), then merges into global once (one lock, short hold time). Invariant: $\sum histogram[i] = n$

Listing 8: Histogram — local compute then single merge under mutex

```

1  #include <stdio.h>
2  #include <pthread.h>
3
4  #define HIST_SIZE 10
5  #define N 9
6  #define SUB_ARRAY_COUNT 4
7  #define SUB_ARRAY_DIM 3
8  #define RANGE 10
9
10 int input[N] = {3,1,4,1,5,9,2,6,5};
11 int global_hist[HIST_SIZE] = {0};
12 pthread_mutex_t hist_mutex;
13
14 typedef struct { int start; int end; } ThreadArgs;
15
16 void *compute_histogram(void *arg) {
17     ThreadArgs *a = (ThreadArgs*)arg;
18     int local_hist[HIST_SIZE] = {0}; // PRIVATE    no mutex needed here
19     for (int i=a->start; i<a->end; i++)
20         local_hist[input[i]]++; // count locally
21
22     pthread_mutex_lock(&hist_mutex); // lock ONLY for merge
23     for (int k=0; k<HIST_SIZE; k++) global_hist[k] += local_hist[k];
24     pthread_mutex_unlock(&hist_mutex);
25     return NULL;
26 }
27
28 int main() {
29     pthread_mutex_init(&hist_mutex, NULL);
30     ThreadArgs args[2] = {{0,5},{5,9}};
31     pthread_t threads[2];
32     pthread_create(&threads[0], NULL, compute_histogram, &args[0]);
33     pthread_create(&threads[1], NULL, compute_histogram, &args[1]);
34     pthread_join(threads[0], NULL); pthread_join(threads[1], NULL);
35     int sum=0;
36     for(int i=0; i<HIST_SIZE; i++){printf("[%d]=%d\n", i, global_hist[i]); sum+=global_hist[i];}
37     printf("Sum=%d (should be %d)\n", sum, N);
38     return 0;
39 }

```

CPU Scheduling Algorithms (C++)

Core idea: All algorithms answer the same question: “which ready process runs next?” They differ only in the selection rule and whether the current process can be interrupted.

Algorithm	Selection Rule	Preemptive?	Key property
FCFS	Earliest arrival time	No	Simple, convoy effect
SJF	Shortest burst time (arrived)	No	Optimal avg wait (non-preemptive)
Priority	Best priority number (lower=better)	No	Starvation possible
SRTF	Shortest remaining time (any)	Yes	Optimal avg wait (preemptive)
Round Robin	Circular queue, fixed quantum	Yes	Fair, good for time-sharing

Listing 9: CPU Scheduling — all 5 algorithms + Gantt chart printer

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <algorithm>
5  #include <string>
6  using namespace std;
7
8  struct Process { string name; int arrival_time, burst_time, priority; };
9  struct Slot { string name; int duration; };
10
11 // Prints the Gantt chart from a list of (process, duration) slots
12 void printGantt(const vector<Slot> &slots, const string &title) {
13     cout << "\n[ " << title << " ]\n=== Gantt Chart ===\n";
14     for (auto &s:slots){cout<<" "; for(int i=0; i<s.duration*2+2; i++) cout<<"-";} cout<<"\n";
15     for (auto &s:slots){

```

```

16         cout<<"| "; int pad=s.duration*2-(int)s.name.size()+1;
17         cout<<s.name; for(int i=0;i<pad;i++) cout<<" ";
18     } cout<<"\n";
19     for (auto &s:slots){cout<<" "; for(int i=0;i<s.duration*2+2;i++) cout<<"-";} cout<<"\n";
20     int t=0; cout<<t;
21     for (auto &s:slots){
22         t+=s.duration; int w=s.duration*2+3; string ts=to_string(t);
23         for(int i=0;i<w-(int)ts.size();i++) cout<<" "; cout<<ts;
24     } cout<<"\n===== \n";
25 }
26
27 // 1. FCFS: sort by arrival, run each to completion in order
28 void fcfs(vector<Process> p) {
29     sort(p.begin(),p.end(),[](const Process&a,const Process&b){return a.arrival_time<b.arrival_time
30         ;});
31     vector<Slot> g; int t=0;
32     for (auto &proc:p){if(t<proc.arrival_time) t=proc.arrival_time; g.push_back({proc.name,proc.
33         burst_time}); t+=proc.burst_time;}
34     printGantt(g,"FCFS");
35 }
36
37 // 2. SJF Non-Preemptive: at each decision, pick shortest burst from arrived processes
38 void sjf(vector<Process> p) {
39     int n=p.size(),t=0,done=0; vector<bool> fin(n,false); vector<Slot> g;
40     while(done<n){
41         int best=-1;
42         for(int i=0;i<n;i++) if(!fin[i]&&p[i].arrival_time<=t)
43             if(best==-1||p[i].burst_time<p[best].burst_time) best=i;
44         if(best==-1){t++;continue;}
45         g.push_back({p[best].name,p[best].burst_time}); t+=p[best].burst_time; fin[best]=true; done
46         ++;
47     }
48     printGantt(g,"SJF Non-Preemptive");
49 }
50
51 // 3. Priority Non-Preemptive: same as SJF but compare priority (lower = better)
52 void priority_np(vector<Process> p) {
53     int n=p.size(),t=0,done=0; vector<bool> fin(n,false); vector<Slot> g;
54     while(done<n){
55         int best=-1;
56         for(int i=0;i<n;i++) if(!fin[i]&&p[i].arrival_time<=t)
57             if(best==-1||p[i].priority<p[best].priority) best=i;
58         if(best==-1){t++;continue;}
59         g.push_back({p[best].name,p[best].burst_time}); t+=p[best].burst_time; fin[best]=true; done
60         ++;
61     }
62     printGantt(g,"Priority Non-Preemptive");
63 }
64
65 // 4. SRTF: every time unit, run process with smallest remaining time (preemptive)
66 void srtf(vector<Process> p) {
67     int n=p.size(),t=0,done=0; vector<int> rem(n); for(int i=0;i<n;i++) rem[i]=p[i].burst_time;
68     vector<Slot> g; int last=-1,run_len=0;
69     while(done<n){
70         int best=-1;
71         for(int i=0;i<n;i++) if(rem[i]>0&&p[i].arrival_time<=t)
72             if(best==-1||rem[i]<rem[best]) best=i;
73         if(best==-1){if(last!=-1){g.push_back({p[last].name,run_len});last=-1;run_len=0;} t++;
74             continue;}
75         if(best!=last){if(last!=-1) g.push_back({p[last].name,run_len}); last=best; run_len=0;}
76         rem[best]--; run_len++; t++;
77         if(rem[best]==0) done++;
78     }
79     if(last!=-1) g.push_back({p[last].name,run_len});
80     printGantt(g,"SRTF (Preemptive SJF)");
81 }
82
83 // 5. Round Robin: rotate queue, each process gets at most 'quantum' units per turn
84 void round_robin(vector<Process> p, int quantum=2) {
85     int n=p.size(),t=0,done=0; vector<int> rem(n); for(int i=0;i<n;i++) rem[i]=p[i].burst_time;
86     vector<int> order(n); for(int i=0;i<n;i++) order[i]=i;
87     sort(order.begin(),order.end(),[&](int a,int b){return p[a].arrival_time<p[b].arrival_time;});
88     queue<int> ready; vector<bool> in_q(n,false); vector<Slot> g;
89     for(int idx:order) if(p[idx].arrival_time<=t&&!in_q[idx]){ready.push(idx);in_q[idx]=true;}
90     while(done<n){
91         if(ready.empty()){
92             t++;
93             for(int idx:order) if(!in_q[idx]&&rem[idx]>0&&p[idx].arrival_time<=t){ready.push(idx);
94                 in_q[idx]=true;}

```

```

89         continue;
90     }
91     int cur=ready.front(); ready.pop();
92     int run=min(quantum,rem[cur]);
93     g.push_back({p[cur].name,run}); t+=run; rem[cur]-=run;
94     for(int idx:order) if(!in_q[idx]&&rem[idx]>0&&p[idx].arrival_time<=t){ready.push(idx);in_q[
95         idx]=true;}
96     if(rem[cur]==0) done++; else ready.push(cur);
97 }
98 printGantt(g,"Round Robin (q=2)");
99 }
100
101 int main() {
102     vector<Process> procs={{ "P1",0,5,2},{ "P2",1,4,1},{ "P3",2,2,3},{ "P4",3,6,2},{ "P5",4,3,4}};
103     fcfs(procs); sjf(procs); priority_np(procs); srtf(procs); round_robin(procs,2);
104     return 0;
105 }

```

Deadlock

The 4 Conditions (ALL must hold for deadlock)

Condition	Meaning	How to break
Mutual Exclusion	Resource held by only one process	Make resource shareable (not always possible)
Hold & Wait	Holding one, waiting for another	Request all resources at once before start
No Preemption	Resource can't be forcibly taken	Forcibly reclaim resources if blocked
Circular Wait	Chain: P1 waits P2 waits ... P1	Always acquire resources in fixed order

Resource Allocation Graph (RAG) Cycle Detection

Why DFS: RAG represents hold/wait relationships as a directed graph. A cycle = circular wait = deadlock. DFS with a “currently in path” stack detects back edges (cycles).

Listing 10: RAG — DFS cycle detection. Cycle = deadlock.

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define P 3    // processes: 0=P1, 1=P2, 2=P3
5  #define R 3    // resources: 3=R1, 4=R2, 5=R3
6  #define N (P+R)
7
8  int graph[N][N];    // adjacency matrix
9  int visited[N], in_stack[N];
10
11 int dfs(int node) {
12     visited[node] = in_stack[node] = 1;
13     for (int next=0; next<N; next++) {
14         if (graph[node][next]) {
15             if (!visited[next] && dfs(next)) return 1; // cycle found deeper
16             else if (in_stack[next]) return 1;        // back edge = cycle!
17         }
18     }
19     in_stack[node] = 0;
20     return 0;
21 }
22
23 int detect_cycle() {
24     memset(visited,0,sizeof(visited)); memset(in_stack,0,sizeof(in_stack));
25     for(int i=0;i<N;i++) if(!visited[i] && dfs(i)) return 1;
26     return 0;
27 }
28
29 int main() {
30     memset(graph,0,sizeof(graph));
31     // Assignment edges (Resource -> Process): R1 held by P1, etc.
32     graph[3][0]=1; graph[4][1]=1; graph[5][2]=1;
33     // Request edges (Process -> Resource): P1 wants R2, P2 wants R3, P3 wants R1
34     graph[0][4]=1; graph[1][5]=1; graph[2][3]=1; // circular! -> deadlock
35     printf("%s\n", detect_cycle() ? "DEADLOCK DETECTED" : "No deadlock");
36     return 0;
37 }

```

Banker's Algorithm (Deadlock Avoidance)

Why Banker's: Before granting any resource request, simulate the allocation and check if a “safe sequence” still exists — an order in which all processes can finish using available resources. If no safe sequence exists after granting $\Rightarrow$ deny the request.

Key tables: $\text{Allocation}[i][j] = \text{held}$, $\text{Max}[i][j] = \text{ever needed}$, $\text{Need}[i][j] = \text{Max} - \text{Allocation}$, $\text{Available}[j] = \text{free now}$.

Listing 11: Banker's Algorithm — safety check + resource request

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define P 5
5  #define R 3
6
7  int allocation[P][R]={0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2};
8  int max_d[P][R]    ={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
9  int available[R]    = {3,3,2};
10 int need[P][R];
11
12 int can_run(int i, int work[]) {
13     for(int j=0;j<R;j++) if(need[i][j]>work[j]) return 0;
14     return 1;
15 }
16
17 void safety_check() {
18     // Compute need = max - allocation
19     for(int i=0;i<P;i++) for(int j=0;j<R;j++) need[i][j]=max_d[i][j]-allocation[i][j];
20
21     int work[R], finish[P], seq[P], cnt=0;
22     memcpy(work,available,sizeof(available)); memset(finish,0,sizeof(finish));
23
24     int found;
25     do {
26         found=0;
27         for(int i=0;i<P;i++) {
28             if(!finish[i] && can_run(i,work)) {
29                 for(int j=0;j<R;j++) work[j]+=allocation[i][j]; // pretend it finishes
30                 finish[i]=1; seq[cnt++]=i; found=1;
31             }
32         }
33     } while(found);
34
35     if(cnt==P){
36         printf("SAFE. Sequence: ");
37         for(int i=0;i<P;i++) printf("%d%s",seq[i],i<P-1?" -> ":"\n");
38     } else printf("UNSAFE      deadlock possible!\n");
39 }
40
41 void request(int pid, int req[]) {
42     printf("P%d requests [%d,%d,%d]: ",pid,req[0],req[1],req[2]);
43     for(int j=0;j<R;j++) if(req[j]>need[pid][j]){printf("Exceeds max!\n");return;}
44     for(int j=0;j<R;j++) if(req[j]>available[j]){printf("Must wait.\n");return;}
45     // Tentatively allocate then check safety
46     for(int j=0;j<R;j++){available[j]-=req[j]; allocation[pid][j]+=req[j]; need[pid][j]-=req[j];}
47     safety_check(); // if unsafe, in real OS: rollback above changes
48 }
49
50 int main() {
51     printf("=== Initial State ===\n"); safety_check();
52     int req[]={1,0,2}; request(1, req); // P1 requests (1,0,2)
53     return 0;
54 }

```

Deadlock Detection Algorithm

Why different from Banker's: Banker's is *avoidance* (check before granting). Detection is used when you *allow* deadlock to happen then find who's stuck. No Max table needed — you only check current **Request** (what each process is *waiting for right now*). Processes still unfinished at the end $\Rightarrow$ deadlocked.

Listing 12: Deadlock Detection — finds which processes are deadlocked

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define P 5
5  #define R 3
6

```

```

7  int allocation[P][R]={0,1,0},{2,0,0},{3,0,3},{2,1,1},{0,0,2}};
8  int request[P][R]    ={{0,0,0},{2,0,2},{0,0,0},{1,0,0},{0,0,2}}; // current wait requests
9  int available[R]     = {0,0,0};
10
11  int leq(int a[], int b[]) { for(int j=0;j<R;j++) if(a[j]>b[j]) return 0; return 1; }
12
13  void detect() {
14      int work[R]; memcpy(work,available,sizeof(available));
15      int finish[P];
16      // finish=true for processes with no allocation (not involved in deadlock)
17      for(int i=0;i<P;i++){
18          finish[i]=1;
19          for(int j=0;j<R;j++) if(allocation[i][j]!=0){finish[i]=0; break;}
20      }
21      int found;
22      do {
23          found=0;
24          for(int i=0;i<P;i++){
25              if(!finish[i] && leq(request[i],work)){
26                  for(int j=0;j<R;j++) work[j]+=allocation[i][j]; // release resources
27                  finish[i]=1; found=1;
28                  printf("P%d can complete.\n",i);
29              }
30          }
31      } while(found);
32
33      int dl=0;
34      printf("Deadlocked: ");
35      for(int i=0;i<P;i++) if(!finish[i]){printf("P%d ",i); dl=1;}
36      if(!dl) printf("None");
37      printf("\n");
38  }
39  int main() { detect(); return 0; }

```

Quick Reference Card

Mutex API

```

pthread_mutex_t m;
pthread_mutex_init(&m, NULL);
pthread_mutex_lock(&m); // acquire
// critical section
pthread_mutex_unlock(&m); // release
pthread_mutex_destroy(&m);

```

Thread API

```

pthread_t tid;
pthread_create(&tid, NULL, func, arg);
pthread_join(tid, NULL); // wait for it
// func signature:
void *func(void *arg) { ...; return NULL; }

```

Semaphore API

```

sem_t s;
sem_init(&s, 0, N); // N=slots (0=signal)
sem_wait(&s); // P() -- decrement/block
sem_post(&s); // V() -- increment/wake
sem_destroy(&s);

```

Semaphore vs Mutex

Mutex	Binary (0/1), same thread unlocks
Semaphore	Counter (0..N), any thread posts
Mutex init 1	≡ Semaphore init 1
Semaphore init 0	Used as a signal/relay

Deadlock conditions (ALL 4 required): Mutual Exclusion $\wedge$ Hold & Wait $\wedge$ No Preemption $\wedge$ Circular Wait